

2026-SP 育种模型项目

项目分工说明与从零实现教程

2026-07-28 3R+5S 架构, 从零到交付全流程

目录

- 一、项目 workflows 总览..... 3
 - 1.1 数据流..... 3
 - 1.2 交叉验证线..... 3
 - 1.3 角色总览..... 4
- 二、R1 特征工程架构师 — 完整教程（40%工作量） 5
 - 2.1 第一步：数据清洗..... 5
 - 任务目标..... 5
 - 为什么用 KNN 插值而非其他方法 5
 - 实现代码..... 6
 - 2.2 第二步：特征工程构建..... 6
 - 任务目标..... 6
 - 交互特征（Interaction Features） 6
 - 动态环境指纹特征（Temporal-Dynamic Environmental Fingerprint） 6
 - EESI 极端事件敏感性指数（Extreme Event Sensitivity Index） 7
 - 空间特征..... 7
 - 横向对比：为什么不使用更复杂的方法..... 7
 - 实现代码..... 7

2.3 第三步：特征选择与降维.....	8
任务目标.....	8
步骤一：VIF 筛选（Variance Inflation Factor, 方差膨胀因子）.....	8
步骤二：PCA 降维（Principal Component Analysis, 主成分分析）.....	9
步骤三：Boruta 算法确认.....	9
三、R2 建模与优化工程师 — 完整教程（30%工作量）.....	10
3.1 第一步：基线模型.....	10
3.2 第二步：集成模型.....	10
RF（Random Forest, 随机森林）.....	11
XGBoost（Extreme Gradient Boosting, 极端梯度提升）.....	11
3.3 第三步：超参数优化.....	12
3.4 第四步：QRF 不确定性量化.....	13
3.5 第五步：NSGA-II 多目标优化.....	14
四、R3 模型解释与交付工程师 — 完整教程（20%工作量）.....	14
4.1 SHAP 全链路分析.....	14
4.2 鲁棒性验证.....	15
4.3 代码交付.....	15
五、S1 与 S2 交叉验证角色 — 完整教程（合计 10%工作量）.....	16
5.1 S1 特征审计员.....	16
5.2 S2 模型验证员.....	16
六、S4-S6 独立支撑角色.....	17
6.1 S4 文献调研与论文写作.....	17
6.2 S5 主体图设计师.....	17
6.3 S6 附录图设计师.....	18
七、总参考文档目录.....	18

一、项目 workflows 总览

2026-SP 项目的目标是构建一个基于环境指纹（Environmental Fingerprint）和机器学习（Machine Learning, ML）的冬小麦产量预测与管理优化系统。工作流分为三条线：核心编程线（R1 至 R3）、交叉验证线（S1 至 S2）、并行产出线（S4 至 S6）。

1.1 数据流

数据搜集将数据交付给 R1。R1 对原始 73 个环境变量进行清洗、特征工程构建、降维和筛选，产出最终特征矩阵 X_{final} （234 行网络乘以 d 列特征，其中 d 为目标维度数，预期 25 至 35 列）。R2 接收 X_{final} ，训练多个机器学习模型（OLS, Ordinary Least Squares 普通最小二乘回归、Ridge 岭回归、LASSO, Least Absolute Shrinkage and Selection Operator 最小绝对收缩和选择算子、RF, Random Forest 随机森林、XGBoost, Extreme Gradient Boosting 极端梯度提升、LightGBM, Light Gradient Boosting Machine 轻量梯度提升机），通过 Optuna（超参数优化框架）搜索最优参数，使用分位数回归森林 QRF（Quantile Regression Forest）输出预测不确定性，最终使用非支配排序遗传算法第二代 NSGA-II（Non-dominated Sorting Genetic Algorithm II）进行管理优化。R3 接收 R2 的最佳模型，使用 SHAP（SHapley Additive exPlanations, 沙普利加性解释）进行全局和局部解释、Bootstrap（自助重抽样）稳健性验证、整合全部代码和文档打包为 GitHub Release（发布版本）。

1.2 交叉验证线

S1 与 R1 并行：S1 使用与 R1 不同的统计库和方法（statsmodels 而非 sklearn）独立验证 R1 的特征选择决策。S2 与 R2 并行：S2 使用 R2 未使用的模型（SVR, Support Vector Regression 支持向量回归、GBM, Gradient Boosting Machine 梯度提升机）独立检验 R2 的模型性能。当 S1 或 S2 发现与对应 R 不一致的结果时，提交审计报告触发复查流程。

1.3 角色总览

代号	角色	工作量	核心职责	输入	输出
R1	特征工程架构师	40%	数据清洗、特征构建、降维、特征选择	pinggu_environmental_data.csv (234x73)	X_final.csv (234xd)
R2	建模与优化工程师	30%	基线、集成模型、超参数优化、QRF、NSGA-II	X_final.csv	best_model.pkl + predictions.csv
R3	模型解释与交付工程师	20%	SHAP 分析、Bootstrap 验证、代码文档、Release	best_model.pkl + X_final.csv	SHAP 报告+GitHub Release
S1	特征审计员	10%合计	独立 VIF/PCA/泄露检查	同 R1 输入	feature_audit_report.md
S2	模型验证员		独立模型 /LOYO/噪声/极端输入	X_final.csv	model_validation_report.md
S4	文献与写作		IMRaD 论文全流程	所有 R+S 产物	Manuscript.docx
S5	主体图设计师		Fig 1 至 7, 600 DPI TIFF	R1/R2/R3 输出数据	Fig1-7.tif
S6	附录图设计+规范		Fig S1 至 S12, 300 DPI PNG + 风格表	R1/R2/R3 输出数据	FigS1-12.png + style.mplstyle

二、R1 特征工程架构师 — 完整教程（40%工作量）

2.1 第一步：数据清洗

任务目标

读取 `pinggu_environmental_data.csv`（234 行、73 列），处理缺失值和异常值，统一量纲。当前数据唯一的问题：地形三列（`elevation_m` 海拔高度、`slope_deg` 坡度角度、`aspect_deg` 坡向角度）各有 18 个 NaN（Not a Number, 缺失值，占比 7.7%），出现在边缘网格——因为 SRTM（Shuttle Radar Topography Mission, 航天飞机雷达地形测绘任务）数据在平谷区边界以外的网格没有高程记录。

为什么用 KNN 插值而非其他方法

KNN（K-Nearest Neighbors, K 近邻插值）选择 K=5 个空间相邻网格的平均值来填充缺失值，这在地理数据中比中位数填充或均值填充更合理——因为海拔在空间上是连续变化的，相邻网格的海拔比全区域均值更接近真实值。

对比以下四种方案：

- (1) 方案 A（均值/中位数填充）：用全区域均值填充 18 个缺失网格。问题：平谷区海拔从 15 米到 898 米变化巨大，均值约 200 米，对于位于山区的边缘网格（实际海拔可能 500 米以上）会产生数百米的误差。不推荐。
- (2) 方案 B（直接剔除）：删除含 NaN 的 18 行，还剩 216 个网格。问题：减少了 7.7% 的样本量，且被剔除的恰好是海拔最高、环境最独特的山区网格，会系统性地削去数据中的极端环境信号。不推荐。
- (3) 方案 C（KNN 插值，K=5）：用距离最近的 5 个网格的平均值填充。保留了全部 234 个网格，插值误差在均质区域（坡度平缓的平原）极小，在异质区域（山脚过渡带）可接受。推荐。
- (4) 方案 D（克里金插值 Kriging Interpolation）：统计学最优的插值方法，考虑了空间自相关性。精度高于 KNN 但实现复杂。如果后续审稿人对 KNN 插值提出质疑，可以升级为克里金。暂不采用 Kriging——因为仅 18 个 NaN，KNN 足够。

实现代码

```
import pandas as pd; from sklearn.impute import KNNImputer; df =  
pd.read_csv("pinggu_environmental_data.csv"); imputer = KNNImputer(n_neighbors=5);  
df_filled = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)。IQR 乘以 3 异常值  
检测:  $Q1 = df.quantile(0.25)$ ;  $Q3 = df.quantile(0.75)$ ;  $IQR = Q3 - Q1$ ;  $outliers = (df < Q1 - 3*IQR) | (df > Q3 + 3*IQR)$ 。输出: cleaned_data.csv (234 行、71 列, 零缺失值)。
```

2.2 第二步: 特征工程构建

任务目标

从现有的 71 个变量出发, 构造新的衍生特征 (交互项、聚合项、极端指数), 使特征集从 71 列扩展到约 85 列以上。这一步是 R1 工作中最具创造性的部分——好的特征工程比好的模型更重要。

交互特征 (Interaction Features)

含义: 两个变量同时偏高或偏低时对产量产生的联合效应, 超出各自独立效应的简单相加。构建逻辑: 温度乘以降水——4 月 (拔节期) 平均温度乘以 4 月降水量, 7 月 (灌浆期) 平均温度乘以 7 月降水量。这两个时期的小麦对水热组合最为敏感。GDD

(Growing Degree Days, 生长期日——温度超过作物生长阈值后的累积热量单位) 乘以土壤保水能力 (粉粒 silt_pct 加粘粒 clay_pct 之和): 热量充足但保水差的土壤与热量充足保水好的土壤, 其产量潜力完全不同。技术上: 在 Pandas 中直接 $df["temp4_x_prec4"] = df["tavg_4"] \times df["prec_4"]$ 。

动态环境指纹特征 (Temporal-Dynamic Environmental Fingerprint)

这是相对于 Bai et al.(2024)的核心改进——他们使用 30 年气候均值构建静态指纹, 忽略了年际变异。我们新增三类:

(1) 月均温的变异系数 CV (Coefficient of Variation, 变异系数等于标准差除以均值, 衡量相对波动程度) ——公式为 $CV_Tavg = \text{std}(tavg_1...tavg_12) / \text{mean}(tavg_1...tavg_12)$ 。月降水的变异系数 CV_Prec 同理。

(2) 高温胁迫月数 (N_heat_stress_months)：统计 12 个月中有多少个月的月均温超过特定阈值（如 25 摄氏度）。

(3) 春季霜冻风险 (frost_risk_spring)：基于最低温代理变量，统计 3 月至 4 月期间低于特定阈值的频率。

EESI 极端事件敏感性指数 (Extreme Event Sensitivity Index)

(1) Tmax_extreme_intensity：通过 WorldClim 的 bio5（最暖月最高温）与 bio10（最暖季均温）的差值，近似估算极端高温强度。

(2) Drought_spell_indicator：基于月降水序列，统计连续低于月降水第 10 百分位数的月数。

(3) Frost_spring_indicator：基于 bio 变量的代理指标，统计 3 至 4 月期间出现低于特定温度阈值的频率。

空间特征

dist_to_center：每个网格到平谷区几何中心（约 116.95°E, 40.25°N）的欧氏距离。意义：距离中心越远则越接近山区，呈现产量梯度信号。lat_effect 和 lon_effect：纬度和经度的平方项，捕捉空间梯度的非线性部分。

横向对比：为什么不使用更复杂的方法

(1) 自动特征工程（如 AutoFeat、tsfresh）：适合超高维数据（数百列以上），我们的 71 列不算高维，人工构建的交互特征具有明确的农学含义，可解释性强于黑箱自动生成。暂不采用。

(2) 深度学习自动特征（如 Autoencoder 自动编码器提取的潜在表示）：特征不可单独解释，论文无法开展 SHAP 分析。与我们的可解释性目标冲突。不采用。

(3) GAMs (Generalized Additive Models, 广义加性模型) 自动非线性变换：可以用 splines（样条函数）自动寻找最优非线性变换，但增加了模型复杂度。仅在 R2 发现特征线性度差时作为备选。目前不采用。

实现代码

```
df["temp4_x_prec4"] = df["tavg_4"] * df["prec_4"]; df["temp7_x_prec7"] = df["tavg_7"] *  
df["prec_7"]; df["GDD_x_soil_water"] = df["GDD_season"] * (df["silt_pct"] + df["clay_pct"]);
```

```
cols_tavg = [co for co in df.columns if co.startswith("tavg_")]; df["CV_tavg"] =  
df[cols_tavg].std(axis=1) / df[cols_tavg].mean(axis=1)。输出：engineered_features.csv（234  
行，85 列以上）。
```

2.3 第三步：特征选择与降维

任务目标

从 85 列以上特征中筛选出 25 至 35 个最优特征组成最终建模矩阵。核心原则：特征越少，模型越稳健——多余的特征不仅降低计算效率，还会引入噪声和多重共线性（Multicollinearity——多个特征高度相关，导致模型系数估计不稳定）。

步骤一：VIF 筛选（Variance Inflation Factor, 方差膨胀因子）

目的：剔除多重共线性严重的特征。VIF 衡量一个特征能被其他特征线性解释的程度：VIF 等于 1 表示完全独立，VIF 为 5 至 10 表示中等共线，VIF 大于 10 表示严重共线。我们采取迭代剔除策略：计算所有特征的 VIF 值，找出 VIF 最大的特征，如果 VIF 大于 10 则剔除该特征，重新计算所有剩余特征的 VIF，重复直到所有 VIF 小于 10。

为什么 VIF 而不是 Pearson 相关系数：Pearson 相关系数（Pearson Correlation Coefficient）只能检测两个变量之间的线性关系，无法检测多个变量的共线性。例如，变量 A 可以由变量 B 和变量 C 的线性组合完美表示，但 A 与 B、A 与 C 的单一相关系数可能都不高。VIF 通过回归全部变量来检测这种多变量共线性。我们两者都做：Pearson 用于快速排除完美线性对（R 大于 0.95），VIF 用于精细筛除多变量共线。

为什么阈值选 10：这是统计学标准惯例（Hair et al., 2010）。更严格的标准是 VIF 小于 5（适用于需要极高系数稳定性的场景），更宽松的是 VIF 小于 15（适用于纯预测目标）。我们选择 VIF 小于 10 作为折中——既有统计保证，又不至于过度削减特征。

实现：from statsmodels.stats.outliers_influence import variance_inflation_factor; while True: vif = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]; if max(vif) <= 10: break; drop_idx = vif.index(max(vif)); X = X.drop(X.columns[drop_idx], axis=1)

步骤二：PCA 降维（Principal Component Analysis, 主成分分析）

目的：将高维特征空间映射到低维正交空间，保留最大方差的同时消除共线性。PCA 的数学本质是特征值分解——找到数据方差最大的方向（第一主成分 PC1），然后在与 PC1 正交的方向中找方差第二大的（PC2），依此类推。

使用方式：我们对 PCA 和 VIF 两种路径都做，对比最终模型性能后选择。PCA 路径：将 85 列以上特征标准化（StandardScaler，均值 0 方差 1）后做 PCA，保留累计解释方差超过 95%所需的前 k 个主成分（预期 k 约 20 至 30）。VIF 路径：直接对 85 列以上特征做 VIF 筛选，保留约 25 至 35 个原始特征。两种路径产出的特征矩阵分别训练模型，比较空间交叉验证的 R 平方值，选择更好的方案。

为什么用 PCA 保留 95% 方差而非固定维度数：固定维度数（如强制取前 20 个主成分）可能丢失重要信息（如果前 20 个主成分只解释了 80% 方差），或保留噪声（如果前 20 个主成分解释了 99% 方差）。以 95% 方差为阈值是数据自适应的——在信息充分的区域不需要过度降维，在信息稀疏的区域自动保留更多维度。

步骤三：Boruta 算法确认

目的：作为特征选择的"终审"步骤。Boruta 是一种基于随机森林的特征选择算法：它为每个真实特征创建一个"影子特征"（Shadow Feature——将真实特征的值随机打乱得到的对比特征），然后比较真实特征和影子特征对模型的重要性。如果真实特征的重要性显著高于其影子特征（通过统计检验确认），则标记为"已确认"（confirmed）；如果与影子特征无显著差异，标记为"暂定"（tentative）；如果显著低于影子特征，标记为"拒绝"（rejected）。

为什么 Boruta 而非简单的"重要性排序取 Top-25"：简单排序的问题是——重要性排序受随机种子影响，不同随机种子可能给出不同的 Top-25 列表。Boruta 通过统计检验确认每个特征是否"真正重要"，排除了随机波动的影响。

实现：from boruta import BorutaPy; from sklearn.ensemble import RandomForestRegressor; rf = RandomForestRegressor(n_jobs=-1, random_state=42); boruta = BorutaPy(rf, n_estimators="auto", perc=100, random_state=42); boruta.fit(X.values, y.values);

`confirmed = X.columns[boruta.support_].tolist()`。输出：`X_final.csv`（234 行乘以 `d` 列，`d` 为 25 至 35）+ `feature_selection_report.csv`。

三、R2 建模与优化工程师 — 完整教程（30%工作量）

3.1 第一步：基线模型

任何机器学习项目的第一步都是建立基线——知道"最简单的模型能做到什么程度"后，才能判断复杂模型"值得增加多少复杂度"。三个基线模型按复杂度递增排列。

(1) OLS（Ordinary Least Squares, 普通最小二乘回归）：多元线性回归。不需调参，直接拟合。目标：给出纯线性假设下的预测精度下界。预期 R^2 平方约 0.50 至 0.60。

(2) Ridge 回归（岭回归）：在线性回归的目标函数中加入 L2 正则化项（L2 Regularization——回归系数平方和乘以惩罚系数 α ），惩罚系数通过交叉验证选择。优点：对多重共线性稳健。

(3) LASSO 回归（Least Absolute Shrinkage and Selection Operator, 最小绝对收缩和选择算子）：使用 L1 正则化（L1 Regularization——回归系数绝对值之和乘以惩罚系数 α ），可以将不重要特征的系数压缩为零，同时完成特征选择和预测。优点：自动特征选择。

对比：为什么不用 Elastic Net（弹性网络——L1 和 L2 组合）：Elastic Net 是 Ridge 和 LASSO 的混合，需要两个超参数（ α 和 L1/L2 比例）。对于 25 至 35 维特征，双参数搜索的收益通常不如单参数的 Ridge 或 LASSO。如果特征维度降低后仍有共线性问题，再考虑 Elastic Net。目前不优先使用。

实现：`from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV; from sklearn.model_selection import cross_val_score`。所有模型用 5 折空间分块交叉验证评估。

3.2 第二步：集成模型

集成模型（Ensemble Models）是 R2 的核心——将多个"弱学习器"（单个决策树）组合成"强学习器"（随机森林/梯度提升）。集成方法分为两类：Bagging（Bootstrap

Aggregating, 自助聚合, 并行训练多个独立模型后投票/平均) 和 Boosting (提升, 串行训练模型, 每个新模型重点关注前一个模型预测错误的样本)。

RF (Random Forest, 随机森林)

属于 Bagging 类。在 Bootstrap 抽样 (从原始数据有放回地抽样形成训练子集) 的每个子集上训练一棵未剪枝决策树, 每棵树在每个节点分裂时只考虑随机选取的 m 个特征 (m 约等于总特征数的平方根)。超参数: `n_estimators` (树的数量, 100 至 1000); `max_depth` (最大深度, 5 至 20); `min_samples_split` (节点分裂最小样本数, 5 至 20); `max_features` (每次分裂考虑的特征数, `sqrt` 或 `log2`)。RF 的一个重要特性是 OOB (Out-of-Bag 袋外误差——Bootstrap 未被抽中的约 37% 样本构成天然的验证集)。优点: 不需要单独的验证集即可评估泛化性能。

为什么 RF 作为基线集成模型而非 AdaBoost (Adaptive Boosting 自适应提升): RF 天然抗过拟合 (每棵树独立在不同的 Bootstrap 子集上训练, 树之间的低相关性减小了方差)。AdaBoost 更容易对噪声样本过拟合。在农业产量数据中, 个别网格的极端环境值相当于噪声——RF 比 AdaBoost 更能抵抗这种噪声。

XGBoost (Extreme Gradient Boosting, 极端梯度提升)

属于 Boosting 类, 目前表格数据 (Tabular Data, 结构化数据——行列格式的数据表) 中最强的模型之一。核心改进包含三项: 第一, 在目标函数中加入正则化项 (叶子节点数量加叶子权重平方和), 同时优化预测精度和模型复杂度; 第二, 使用二阶泰勒展开 (Taylor Expansion——用函数的导数信息近似函数值) 近似损失函数, 比传统梯度提升的一阶方法更精确地找到最优分裂点; 第三, 列块结构 (Column Block)——数据按列压缩存储, 极大加速分裂搜索。

关键超参数及调参逻辑: `learning_rate` (学习率, 控制每棵树对最终预测的贡献比例, 0.01 至 0.3), 小的学习率加大的树数等于更平滑的学习但更慢, 建议从 0.1 开始。`max_depth` (树的最大深度, 3 至 10), 对于农业环境数据浅树 (3 至 5) 通常优于深树——环境因子对产量的影响以叠加效应为主, 高阶交互较少。`subsample` (子样本比例, 0.6 至 1.0), 每棵树只用部分数据训练——这是 XGBoost 内置的 Bagging 降低方差, 建议 0.8。

`colsample_bytree`（列采样比例，0.6 至 1.0），每棵树只用部分特征进一步降低树间相关性。
`n_estimators`（树的数量，100 至 1000），配合早停（Early Stopping——验证集性能连续 N 轮不提升则停止训练，此处 N 等于 50）使用，不需要手动设定。

为什么要包括 LightGBM（Light Gradient Boosting Machine）作为对比：LightGBM 使用 leaf-wise（按叶子生长，每次选择使损失下降最大的叶子进行分裂）而非 XGBoost 的 level-wise（按层生长，一次性分裂当前层的所有叶子）。Leaf-wise 在相同树数下通常达到更低的损失——但更容易过拟合小数据集。在 234 个网格数据上，两者性能差异是检验模型稳健性的重要指标。如果两个模型给出高度一致的空间预测模式，则模型结论更可信。

实现代码：from xgboost import XGBRegressor; from lightgbm import LGBMRegressor;
from sklearn.ensemble import RandomForestRegressor。RF:
RandomForestRegressor(n_estimators=500, max_depth=10, min_samples_split=10,
max_features="sqrt", oob_score=True)。XGBoost: XGBRegressor(n_estimators=500,
learning_rate=0.1, max_depth=5, subsample=0.8, colsample_bytree=0.8,
early_stopping_rounds=50)。LightGBM: LGBMRegressor(n_estimators=500, learning_rate=0.1,
max_depth=5, subsample=0.8, colsample_bytree=0.8)。所有模型统一使用空间分块 5 折交叉验证。

3.3 第三步：超参数优化

为什么使用 Optuna 而非 GridSearchCV（网格搜索交叉验证）或 RandomizedSearchCV（随机搜索交叉验证）：

- (1) GridSearchCV（网格搜索）：穷举所有预设参数组合。假设 6 个超参数，每个 5 个候选值，需要训练 5 的 6 次方等于 15625 个模型。不可行。
- (2) RandomizedSearchCV（随机搜索）：从参数空间中随机采样 N 组组合。比 GridSearchCV 高效，但"随机"意味着可能在最优参数附近停止搜索——没有利用已评估参数的信息。
- (3) Optuna（贝叶斯优化框架——推荐）：使用 TPE（Tree-structured Parzen Estimator, 树结构帕尔森估计器——一种基于历史试验结果的概率模型）智能选择下一组参数。每评估一组参数后，TPE 更新其对"好参数"和"坏参数"分布的估计，下一组参数从"好参数"分布中采样。相比

RandomSearchCV, Optuna 通常在更少试验次数内找到更好的参数——对于 150 次试验, Optuna 通常比 RandomSearch 找到的参数更好 10 至 20%。

实现: `import optuna; def objective(trial): params = {"n_estimators": trial.suggest_int("n_estimators", 100, 1000), "max_depth": trial.suggest_int("max_depth", 3, 10)}; model = XGBRegressor(**params); score = cross_val_score(model, X, y, cv=spatial_cv).mean(); return score; study = optuna.create_study(direction="maximize"); study.optimize(objective, n_trials=150)。`

3.4 第四步: QRF 不确定性量化

为什么需要不确定性量化: 一个点预测 (如"该网格产量为 5.2 吨/公顷") 不能告诉使用者这个预测有多可靠。如果两个网格的点预测都是 5.2 吨/公顷, 但一个网格的 90% 预测区间 PI (Prediction Interval) 宽度为 0.5 吨/公顷 (精确), 另一个为 3.0 吨/公顷 (模糊), 决策含义完全不同。QRF (Quantile Regression Forest, 分位数回归森林) 是随机森林的扩展——不预测条件均值, 而是预测条件分位数 (5%、50% 即中位数、95%)。

为什么 QRF 而非其他不确定性方法:

- (1) 方案 A (高斯过程 Gaussian Process): 理论上最优的不确定性量化方法, 同时给出预测值和后验方差。但计算复杂度为 $O(N^3)$, 对于 234 个网格训练时间约数秒, 但对于 N 个虚拟年份的 Bootstrap 重采样 (N 等于 1000) 则不可行。且 GP 需要手动选择核函数 (Kernel Function——定义数据点间相似性的函数), 选择不当会严重影响不确定性估计。我们仅在单个网格级别做 GP 作为对比。
- (2) 方案 B (蒙特卡洛 Dropout, Monte Carlo Dropout): 在推理时保持 Dropout 开启, 多次前向传播得到预测样本集, 从样本集中估计不确定度。仅适用于神经网络, 不能用于 XGBoost。不适用。
- (3) 方案 C (QRF——推荐): 每个网格输出三个值 (5%、50%、95% 分位数)。计算 PIW (Prediction Interval Width, 预测区间宽度等于 $P95 - P05$)。空间化 PIW——绘制每个网格的 PIW 在地图上的着色图——识别模型"知道自己在预测什么"和"不知道"的区域。额外好处: 中位数 (P50) 通常比均值更稳健——不受少数极端预测值影响。

实现：from skgarden import RandomForestQuantileRegressor（或使用 scikit-learn 的 QuantileRegressor 的 RF 适配版）。rfq = RandomForestQuantileRegressor(n_estimators=500, max_depth=10); rfq.fit(X, y); p05 = rfq.predict(X, quantile=5); p50 = rfq.predict(X, quantile=50); p95 = rfq.predict(X, quantile=95); PIW = p95 - p05。输出：predictions_with_uncertainty.csv（234 行乘以 {lon, lat, P05, P50, P95, PIW}）。

3.5 第五步：NSGA-II 多目标优化

目的：找到施肥量和灌溉量的最优组合方案，同时最大化产量和最小化环境成本（氮肥使用量）。单目标优化（如 Grid Search 在 N 肥和灌溉的二维网格上逐一搜索产出最高产量的组合）只有一个最优解——但它忽略了"产量最大化"和"环境最小化"之间的冲突。NSGA-II 的非支配排序（Non-dominated Sorting）找到 Pareto 前沿——即在不降低产量目标的前提下，无法进一步降低环境成本的所有解的集合。

实现简要：from pymoo.algorithms.moo.nsga2 import NSGA2; from pymoo.optimize import minimize。定义问题：两个决策变量（N_rate 施氮量, irrigation 灌溉量），两个目标函数（yield 由 XGBoost 预测，environmental_cost 等于 N_rate 加 irrigation 的加权和）。运行优化后得到 Pareto 前沿散点——这些散点构成了一条"最优权衡曲线"。

四、R3 模型解释与交付工程师 — 完整教程（20%工作量）

4.1 SHAP 全链路分析

SHAP 的核心思想来自博弈论的合作博弈 Shapley 值：把每个输入特征视为"参与者"，把模型预测值视为"合作博弈的总收益"。每个特征的 SHAP 值等于该特征在所有可能的特征组合中的"边际贡献"的平均值。在 ML 中，边际贡献指"当加入该特征时，模型预测值的变化量"。

SHAP beeswarm 图（蜂群图）：Y 轴列出 Top-20 特征，X 轴为 SHAP 值（该特征对预测的边际贡献，单位为吨/公顷）。每个点代表一个网格。点的颜色代表该特征在该网

格的原始值（红色等于高值，蓝色等于低值）。解读逻辑：如果一个特征的红色点全部在右侧（正 SHAP 值），意味着该特征的高值促进产量。这是论文 Fig. 3A 的核心内容。

SHAP 条形图：水平条形，按 mean 绝对值 SHAP value 排序。每个条的长度代表该特征的平均边际贡献幅度。解读：顶部特征对产量的影响最大——无论是正向还是负向。

SHAP 依赖图（Dependence Plot）：X 轴为特征原始值（如海拔从 15 至 898 米），Y 轴为该特征的 SHAP 值。每个散点是一个网格，点的颜色为第二交互特征的原始值。解读：如果 SHAP 值在海拔 500 米附近从正变负——说明存在“海拔阈值”效应。这比简单的相关系数提供了更丰富的非线性信息。

实现：`import shap; explainer = shap.TreeExplainer(model); shap_values = explainer.shap_values(X); shap.summary_plot(shap_values, X, max_display=20); shap.dependence_plot("elevation_m", shap_values, X, interaction_index="soc_dgkg")`。

4.2 鲁棒性验证

Permutation Importance（置换重要性）：随机打乱某个特征的值（破坏该特征与目标变量的关联），记录模型性能下降幅度。打乱最重要特征导致的性能下降越大。对比 SHAP 重要性排序（基于模型内部机制）与 Permutation 排序（基于外部干扰）的一致性——如果两者高度一致（Spearman 秩相关系数大于 0.80），说明 SHAP 结论是稳健的。

Bootstrap 置信区间：从 234 个网格中有放回地随机抽样 234 个（Bootstrap 重采样），重新训练模型，重新计算 SHAP 重要性。重复 1000 次。每个特征得到 1000 个重要性值的分布。计算每个特征的 95% 置信区间。如果某个特征的置信区间不跨零（即下限大于 0 或上限小于 0），说明该特征的重要性是统计显著的。

4.3 代码交付

环境导出：`conda env export > environment.yml`（锁定所有 Python 依赖及其版本号，确保任何人可以在任何机器上一键复现环境）。README.md：包含环境安装（`conda env create -f environment.yml`）、数据准备（从 Data/路径读取）、运行全流程（python

make_all.py)、结果查看 (Outputs/Figures/) 四步指南。所有脚本归档到 Scripts/文件夹, 按数字前缀排序命名 (如 01_clean_data.py, 02_engineer_features.py)。

五、S1 与 S2 交叉验证角色 — 完整教程 (合计 10%工作量)

5.1 S1 特征审计员

S1 不与 R1 共享代码。使用不同工具验证同一结论。具体审计项目:

- (1) 审计一 (独立数据描述): 使用 `df.describe(percentiles=[0.01,0.05,0.25,0.5,0.75,0.95,0.99])` 输出全部变量的分位数表。逐列比对 R1 的输出——如果 R1 报告某变量中位数为 10 而 S1 得到 15, 说明数据在传输中出现了问题。
- (2) 审计二 (独立 VIF 计算): R1 使用 `statsmodels` 的 `variance_inflation_factor`, S1 使用自己编写的 VIF 公式 ($1 / (1 - R^2)$ 函数)。如果两者结果差异超过 1.0, 需要复查。
- (3) 审计三 (交互特征必要性检验): R1 构建的所有交互特征, S1 逐一临时删除该特征后重跑完整流程 (VIF、PCA、RF 训练), 记录 R 平方变化。标记 R 平方变化小于 0.01 的交互特征为“无效交互”——建议 R1 剔除。
- (4) 审计四 (数据泄露检查): 确认 R1 在标准化和 PCA 之前正确拆分了训练集和测试集, 确保测试集信息未泄露到训练集。如果 R1 使用了全量数据的均值和标准差来标准化——即数据泄露——S1 必须要求 R1 改为: 先拆分训练集和测试集, 用训练集的均值和标准差来 transform (变换) 训练集和测试集。

5.2 S2 模型验证员

S2 不与 R2 共享模型代码。使用不同算法和验证策略交叉检验。

- (1) 验证一 (独立模型验证): S2 使用 SVR (Support Vector Regression, 支持向量回归, 核函数等于 RBF 即 Radial Basis Function 径向基函数) 和 GBM (Gradient Boosting Machine) ——这两个模型 R2 的流程中不会使用。在同样的 `X_final` 上用空间分块交叉验证评估。如果 SVR 和 GBM 的 R 平方值与 R2 的 XGBoost 的 R 平方值差异在 0.05 以内——R2 的模型结论可信。

如果差异大于 0.10——说明 R2 的模型可能存在特定假设下的过拟合，需要 S2 在验证报告中标注风险。

- (2) 验证二（LOYO——Leave-One-Year-Out 逐一留年法）：如果年份数据可用，S2 每次留一年的数据作为测试集，用其余 29 年训练。空间交叉验证（空间 CV）与时间交叉验证（LOYO）的对比揭示了模型的泛化能力——如果 LOYO 的 R 平方显著低于空间 CV 的 R 平方（差异大于 0.10），说明模型对时间变化（极端年份）的泛化不如对空间变化的泛化。
- (3) 验证三（噪声特征检验）：S2 构造 3 个纯随机噪声列（`np.random.normal(0,1,234)`），拼接到 `X_final` 中。重新训练 XGBoost。如果噪声特征的 SHAP 重要性进入 Top-15——说明模型可能过拟合了噪音。这需要 R2 降低树的深度或增加正则化。
- (4) 验证四（极端输入测试）：S2 构造极端输入样本——`Tmax` 设置为 50 摄氏度（2022 年华北平原实测记录）、`prec` 设置为 0 毫米（极端干旱）。将这些极端样本输入 R2 的最佳模型。如果模型输出的产量为负值或远超生理上限（15 吨/公顷以上）——说明模型在极端条件下的行为是不合理的，需要在论文中标记“模型在极端条件下的适用性有限”。

六、S4-S6 独立支撑角色

6.1 S4 文献调研与论文写作

S4 全流程平行工作，不阻塞核心流水线。写作策略：提前写好 Methods 的两个核心段落（数据来源、模型描述），等待 R1/R2/R3 产出数字后填入 Results。建议使用 Word（IMRaD 结构——Introduction 引言、Methods 方法、Results 结果、and 连词、Discussion 讨论）。S4 的核心任务是在 Discussion 中将 S1 和 S2 的审计报告整合为“方法学鲁棒性验证”——这本身就是一个创新点：很少有农业 ML 论文公开报告了独立的交叉验证审计。

6.2 S5 主体图设计师

独立制图，不与 R 共享可视化代码。保证一个核心原则：每张图都在讲一个科学故事，而不是展示原始数据。技术栈：`matplotlib`（Python 的绑图库——静态图的主力）、`cartopy`（Python 的地理空间绑图库——处理地图投影和 `shapefile`）、`shap`（直接调用 SHAP 库的内置绑图函数然后自定义样式）。输出全部保存为 600 DPI（dots per inch 每英寸点数）的

TIFF 格式（Tagged Image File Format 标记图像文件格式——无损压缩的专业图像格式，期刊要求的标准格式）。

6.3 S6 附录图设计师

与 S5 并行工作。附录图 300 DPI PNG（Portable Network Graphics 便携式网络图形——压缩但保留细节的通用格式）足够清晰。S6 额外负责创建 style_2026sp.mplstyle 风格表文件——设置 matplotlib 的全局默认参数（全局字体 Arial、全局字号 8-12 磅、全局色条 viridis 加 RdBu、全局去上右边框、全局 DPI、全局图例位置）。这份风格表被 S5 和 S6 共同引用——保证全部 19 张图（7 张主体加 12 张附录）的视觉风格完全一致，形成"视觉品牌"。

七、总参考文档目录

以下是本项目所有成员需要参考的文档总目录，按角色分组。所有管理文档位于 D 盘 2026-SP 文件夹的 ManageFiles 子文件夹内。

文档名称	路径	适用角色	内容说明
项目分工与实现教程	ManageFiles/项目分工与实现教程.docx	所有人	本文档——分工、 workflow、角色教程
项目分工 v4(3R+5S)	ManageFiles/2026-SP_项目分工_v4_3R5S.docx	所有人	角色分工、日程、里程碑
数据获取状态清单	数据获取状态清单.docx	所有人	65 个数据维度的获取状态
数据下载操作手册	数据下载操作手册.md	所有人	各数据源的下载方法
模型数据清单与技术答疑	ManageFiles/模型数据清单与技术答疑.docx	R1,R2,R3,S1,S2	数据清单、环境指纹维度和 Coworker 问答
育种模型技术培	ManageFiles/育种模型项目技术培训手册.docx	所有人	从决策树到

训手册			XGBoost 到 SHAP 的技术原理
创新点与文献缺口分析	ManageFiles/2026-SP_创新点与文献缺口分析.docx	所有人	7个创新点、8个文献缺口、数据不足策略
可视化全案设计	ManageFiles/2026-SP_可视化全案设计_高分论文级.docx	S4,S5,S6	19张图的逐张设计(内容、布局、配色、对标)
参考文献精读与使用指南	ManageFiles/参考文献精读与使用指南.docx	S4,所有人	34篇文献的精读、借鉴部分、引用位置
数据注册表	data_registry.json	R1,R2,S1,S2	JSON格式的数据清单(脚本可读)
README 项目说明	README.md	所有人	GitHub 项目主页——仓库结构和使用说明
AI 编程指南	ManageFiles/README_AI_CODING_GUIDE.md	R1,R2,R3	与 AI 协作编程的指南——提示词和流程说明

7.1 按角色推荐阅读顺序

R1 特征架构师：先读《技术培训手册》技术原理部分、再读《数据清单与技术答疑》、再读本文档 R1 章节、最后参考《可视化全案设计》理解自己的输出如何被可视化。

R2 建模工程师：先读 Bai et al.(2024)论文、《创新点与文献缺口分析》、《技术培训手册》模型部分、再读本文档 R2 章节。

R3 解释工程师：先读 Lundberg & Lee(2017)原始论文、《技术培训手册》SHAP 部分、再读本文档 R3 章节。

S1 特征审计员：使用与 R1 完全相同的参考文档，但用不同的代码实现——参考文档同 R1 列表。

S2 模型验证员：使用与 R2 完全相同的参考文档，但用不同的模型——参考文档同 R2 列表。

S4 论文写作：精读全部《参考文献精读与使用指南》中的 P0 和 P1 优先级别文献（共 14 篇），然后阅读《创新点与文献缺口分析》确定论文贡献声明。

S5 主体图设计：精读《可视化全案设计》的主体图部分加查看 Bai et al.原论文图的布局。

S6 附录图设计：精读《可视化全案设计》的附录图加规范部分、查看 Nature Food 投稿指南的图形规范。